LAPORAN PRAKTIKUM TEKNOLOGI BIG DATA
ARSITEKTUR SISTEM BIG DATA END-TO-END

Disusun oleh :

Misael Feodora Pranata Dauhan	(J0403241145)
Gerry Dimasarya	(J0403241042)
Aryo Ristiawan Machfudz	(J0403241005)
Muhammad Zahran Rabbani	(J0403241077)

TEKNOLOGI REKAYASA PERANGKAT LUNAK

SEKOLAH VOKASI

IPB UNIVERSITY

2026

Studi Kasus

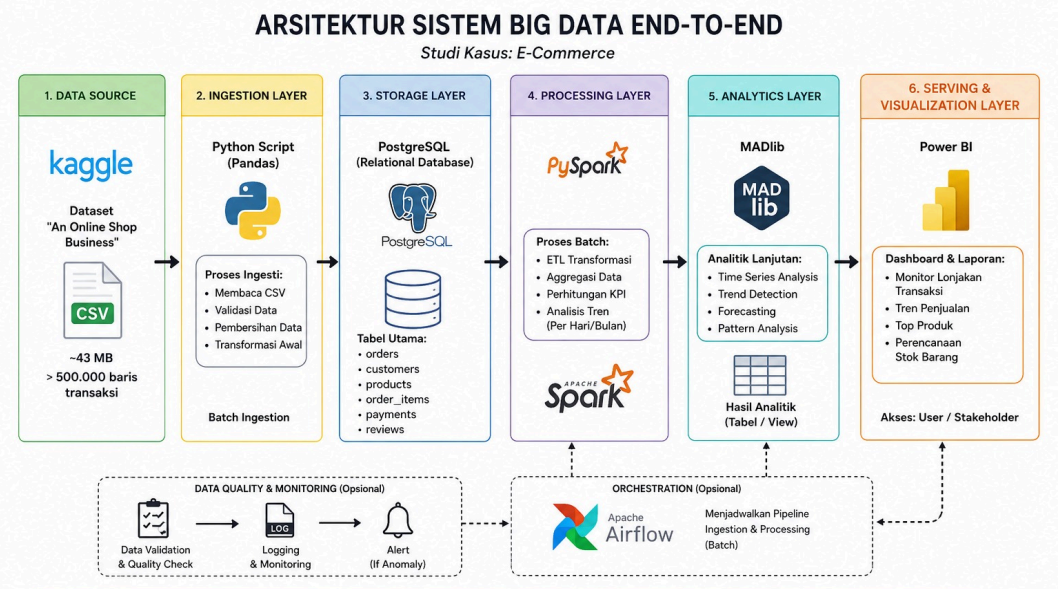
- **Tema:** E-Commerce
- **Sistem:** SIMBA (Sistem Informasi Monitor Batch & Agregasi Stok)

1. **Identifikasi Kebutuhan Sistem**
    a. **Jenis Data yang Digunakan :** Jenis data yang digunakan adalah Structured (Terstruktur).
    b. **Pemrosesan Data :** Bersifat batch, karena pemrosesan dilakukan terhadap rekaman data historis transaksi yang sudah terjadi.
    c. **Estimasi Volume Data :** 43 Megabyte (MB), mewakili lebih dari 500.000 baris transaksi pesanan pelanggan
    d. **Tujuan Analisis Utama :** Memantau lonjakan transaksi berdasarkan hari atau bulan tertentu untuk membantu perencanaan stok barang.
2. **Komponen Arsitektur**

Komponen	**Teknologi yang Dipilih**	**Alasan**
Data Source	Kaggle”"An Online Shop Business"	Dataset berisi data terstruktur (CSV) berupa riwayat transaksi e-commerce skala besar yang ideal untuk analisis tren bisnis.
Data Ingestion	Python Script (Pandas)	Sangat ringan dan praktis untuk mengotomatiskan penarikan *file* CSV mentah secara *batch* tanpa membebani memori pada lingkungan pengembangan lokal.
Storage	PostgreSQL	Data CSV memiliki format terstruktur yang jelas sehingga

		sangat efisien dan logis jika disimpan langsung ke dalam basis data relasional (SQL), yang akan memudahkan kueri manipulasi data analitik tanpa membebani sumber daya komputasi secara berlebih.
Processing	PySpark	Mencegah *bottleneck* performa yang sering terjadi jika pemrosesan skala besar hanya mengandalkan Pandas, sekaligus mempertahankan konsistensi penulisan kode dalam ekosistem Python.
Analytics	MADlib	Sangat menghemat alokasi RAM dan CPU lokal karena data transaksi yang masif tidak perlu diekspor keluar menuju *engine* analitik eksternal untuk mendeteksi tren lonjakan pesanan.
Visualization	Power BI	Memiliki konektor *native* yang terintegrasi langsung ke PostgreSQL, sehingga bisa langsung membaca tabel hasil prediksi analitik dari MADlib tanpa perlu jalur perantara tambahan.

3. **Diagram Arsitektur**



4. Deskripsi Pipeline

a. Alur Data

Menggunakan pendekatan batch processing, alur data sistem SIMBA dimulai dari ekstraksi dataset riwayat transaksi e-commerce (CSV) secara periodik menggunakan skrip Python (Pandas). Data historis ini kemudian disimpan secara terpusat di dalam basis data PostgreSQL, lalu diproses oleh PySpark untuk tahap pembersihan (data cleaning), penanganan nilai kosong, dan agregasi metrik penjualan.

Data yang telah bersih dan terstruktur kemudian dianalisis langsung di dalam database menggunakan Apache MADlib. Sistem mengeksekusi model time series analysis dan deteksi tren untuk memprediksi potensi lonjakan permintaan produk, yang hasilnya langsung dikategorikan menjadi pola pergerakan inventaris (stabil, menurun, atau melonjak).

Pada tahap akhir, wawasan analitik dan metrik distribusi produk divisualisasikan secara langsung melalui dashboard interaktif di Power BI. Laporan ini digunakan sebagai landasan utama bagi pemangku kepentingan untuk memantau performa penjualan dan mengambil keputusan strategis dalam perencanaan stok barang.

b. Integrasi pada Analytics Layer

Pada arsitektur SIMBA, lapisan analitik diintegrasikan secara langsung (in-database analytics) dengan lapisan penyimpanan, sehingga data tidak perlu dipindahkan ke platform eksternal untuk diproses. Apache MADlib dijalankan sebagai ekstensi di atas basis data PostgreSQL yang sama tempat data hasil pembersihan PySpark disimpan, sehingga fungsi-fungsi analitiknya (seperti regresi, time series, dan deteksi tren) dapat dieksekusi langsung dengan perintah SQL pada tabel-tabel transaksi.

Alur integrasinya dimulai ketika PySpark menuliskan kembali (write-back) data yang telah bersih dan teragregasi ke dalam tabel PostgreSQL. MADlib kemudian membaca tabel tersebut untuk menjalankan model analitik prediktif guna mengidentifikasi pola lonjakan transaksi. Hasil keluaran model, berupa tabel prediksi dan kategori tren (stabil, menurun, atau melonjak), disimpan kembali ke dalam basis data yang sama, sehingga siap diakses oleh lapisan visualisasi tanpa memerlukan proses ekspor-impor data tambahan.

c. Serving dan Visualization

Lapisan *serving* dan visualisasi bertugas menyajikan data hasil olahan analitik dari *storage* (PostgreSQL) agar dapat dipahami dan digunakan oleh *stakeholders*. Pada tahap ini, Power BI dihubungkan secara langsung ke basis data PostgreSQL menggunakan *native connector*. Mekanisme ini memungkinkan Power BI membaca *tabel* atau *view* hasil olahan PySpark dan analitik MADlib secara efisien tanpa memerlukan lapisan perantara (*middleware*) tambahan.

Visualisasi disajikan dalam bentuk *dashboard* interaktif dengan beberapa komponen utama, antara lain monitor lonjakan transaksi, tren penjualan, top produk, perencanaan stok. *Dashboard* ini diakses oleh pengguna dan *stakeholders* bisnis sebagai landasan utama dalam memantau performa *e-commerce* serta mengambil keputusan strategis terkait alokasi dan manajemen stok barang.

d. Jenis Processing yang Digunakan

Jenis pemrosesan yang digunakan adalah Batch Processing. Pemrosesan ini dipilih karena sistem melakukan ekstraksi dan analisis

terhadap sekumpulan rekaman data historis transaksi e-commerce yang sudah terjadi secara periodik, bukan memproses data secara real-time.

e. Alasan Pemilihan Arsitektur

Efisiensi Sumber Daya Komputasi Lokal: Penggunaan Python Script (Pandas) untuk penarikan awal sangat ringan dan praktis. Selain itu, penggunaan MADlib memungkinkan proses analitik dilakukan secara langsung di dalam database (in-database analytics), sehingga sangat menghemat alokasi RAM dan CPU karena data yang masif tidak perlu diekspor ke engine eksternal.

Kesesuaian dengan Karakteristik Data: Dataset berupa file CSV yang terstruktur dengan jelas sangat efisien dan logis untuk langsung disimpan ke dalam basis data relasional seperti PostgreSQL. Skalabilitas dan Pencegahan Bottleneck: PySpark dipilih pada lapisan pemrosesan untuk mencegah bottleneck performa yang sering terjadi jika pemrosesan data berskala besar hanya mengandalkan Pandas.

Integrasi yang Mulus (Seamless): Arsitektur ini mempertahankan konsistensi ekosistem penulisan kode (menggunakan Python untuk Pandas dan PySpark). Selain itu, pada lapisan visualisasi, Power BI memiliki konektor native yang terintegrasi langsung ke PostgreSQL sehingga dapat membaca tabel hasil analitik dari MADlib tanpa memerlukan jalur perantara tambahan.